



Siete modelos de concurrency

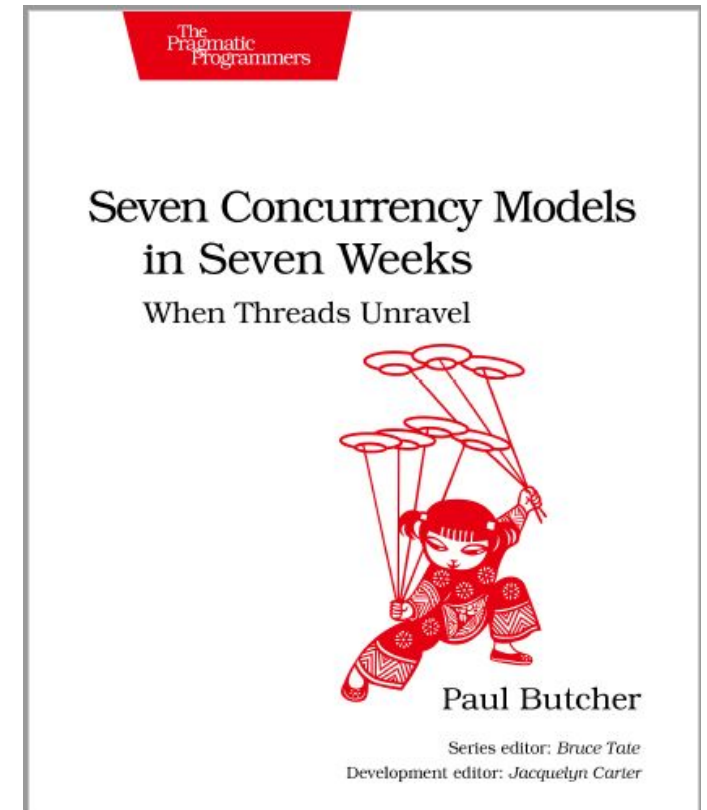
Capitulo 2 día 2

Hilos y cerrojos (Threads and locks)

Seven Concurrency Models

- *Introduction*
- *Threads and locks*
- *Functional programming*
- *The Clojure Way—separating identity and state*
- *Actors*
- *Communicating Sequential Processes*
- *Data parallelism*
- *The Lambda Architecture*

2014



Hilos y cerrojos (Threads and locks)

- La programación con hilos y cerrojos es como un vehículo Ford modelo T. Puede llevarnos del punto A al punto B, pero es primitivo, difícil de conducir, poco confiable –incluso peligroso- comparado con nuevas tecnologías.
- Olvidando un poco sus problemas, la programación con hilos y cerrojos aún se utilizan para escribir mucho software concurrente y como soporta otras tecnologías concurrentes, es bueno estudiarla.



Algo muy simple

- Los hilos y los cerrojos (Locks) son poco más que una formalización de lo que subyace el hardware realmente lo hace. Esa es su gran fuerza y, a la vez, su gran debilidad.
- Debido a que son tan simples, casi todos los lenguajes los admiten de una forma u otra, e imponen muy pocas restricciones sobre lo que se puede lograr a través de su uso.
- Pero tampoco proporcionan casi ninguna ayuda al pobre programador, haciendo que los programas sean muy difíciles de acertar en primer lugar e incluso más difícil de mantener.
- En el capítulo usan el lenguaje Java
 - Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java concurrency in practice*. Addison-Wesley Professional.
 - Urma, R.-G., Fusco, M., & Mycroft, A. (2018). *Modern Java in action: Lambdas, streams, functional and reactive programming*. Manning Publications.

Actividades propuestas para cada día

- Day 1: Mutual Exclusion and Memory Models
 - [Día 1: Exclusión mutua y modelos de memoria]
- Day 2: Beyond Intrinsic Locks
 - [Día 2: Más allá de los cerrojos intrínsecos]
- Day 3: On the Shoulders of Giants
 - [Día 3: Sobre los hombros de gigantes]



Día 2: Más allá de los cerrojos intrínsecos

- El día 1 cubrió la clase Thread de Java y los bloqueos intrínsecos integrados en cada objeto de Java. Durante mucho tiempo, esto fue prácticamente todo el soporte que proporcionó Java para la programación concurrente. Java 5 cambió todo eso con la introducción de java.util.concurrent. Hoy veremos los **mecanismos de bloqueo mejorados** que proporciona.
- los cerrojos intrínsecos son útiles pero limitados.
 - NO hay forma de interrumpir un hilo que está bloqueado como resultado de intentar adquirir un cerrojo intrínseco [¡No se deja hablar!].
 - NO hay forma de tener un tiempo de espera [time out] al intentar adquirir un cerrojo intrínseco. [¡El paso del tiempo no le importa!]
 - Existe exactamente una forma de adquirir un cerrojo intrínseco: un cerrojo *synchronized*.

```
synchronized(object) {  
    <<use shared resources>>  
}
```



Más allá de los cerrojos intrínsecos

- los cerrojos intrínsecos son útiles pero limitadas.
 - Esto significa que la adquisición y liberación de cerrojos deben tener lugar en el mismo método y deben estar estrictamente anidadas. Tenga en cuenta que declarar un método como sincronizado es solo “azúcar” sintáctico para rodear el cuerpo del método con lo siguiente:

```
synchronized(this) {  
    <<method body>>  
}
```

- **ReentrantLock**, otra manera de hacerlo, nos permite trascender estas restricciones al proporcionar métodos explícitos de bloqueo y desbloqueo en lugar de usar `synchronized`.
- Antes de que veamos cómo mejora los bloqueos intrínsecos, a continuación se muestra como `ReentrantLock` se puede usar como un reemplazo directo de `synchronized`:



ReentrantLock es amigable [se puede interrumpir], se deja hablar, el paso del tiempo le importa y se puede usar varias veces.



Más allá de los cerrojos intrínsecos

```
Lock lock = new ReentrantLock();  
lock.lock();  
try {  
    <<use shared resources>>  
} finally {  
    lock.unlock();  
}
```

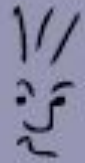
- El try ... finally es una buena práctica para garantizar que el bloqueo siempre se libere, sin importar lo que suceda en el código que protege el lock.
- Ahora veamos cómo elimina las restricciones de los bloqueos intrínsecos.

Bloqueo NO interrumpible

- Debido a que un hilo que está bloqueado en un bloqueo intrínseco no es interrumpible, no tenemos forma de recuperarnos de un punto muerto. Podemos ver esto con un pequeño ejemplo que fabrica un interbloqueo [deadlock] y luego intenta interrumpir los hilos...
- Este programa se bloqueará para siempre [deadlock]; la única forma de salir es interrumpir [kill] la JVM que lo ejecuta.



```
public class Uninterruptible {  
    public static void main(String[] args) throws InterruptedException {  
        final Object o1 = new Object(); final Object o2 = new Object();  
        Thread t1 = new Thread() {  
            public void run() {  
                try {  
                    synchronized(o1) {  
                        Thread.sleep(1000);  
                        synchronized(o2) {}  
                    }  
                } catch (InterruptedException e) { System.out.println("t1 interrupted"); }  
            }  
        };  
        Thread t2 = new Thread() {  
            public void run() {  
                try {  
                    synchronized(o2) {  
                        Thread.sleep(1000);  
                        synchronized(o1) {}  
                    }  
                } catch (InterruptedException e) { System.out.println("t2 interrupted"); }  
            }  
        };  
        t1.start(); t2.start();  
        Thread.sleep(2000);  
        t1.interrupt(); t2.interrupt();  
        t1.join(); t2.join();  
    }  
}
```



Joe asks:

Is There Really No Way to Kill a Deadlocked Thread?

You might think that there has to be *some* way to kill a deadlocked thread. Sadly, you would be wrong. All the mechanisms that have been tried to achieve this have been shown to be flawed and are now deprecated.^a

The bottom line is that there is exactly one way to exit a thread in Java, and that's for the `run()` method to return (possibly as a result of an `InterruptedException`). So if your thread is deadlocked on an intrinsic lock, you're simply out of luck. You can't interrupt it, and the only way that thread is ever going to exit is if you kill the JVM it's running in.

a. <http://docs.oracle.com/javase/1.5.0/docs/guide/misc/threadPrimitiveDeprecation.html>

Bloqueo interrumpible



- Sin embargo, existe una solución. Podemos reimplementar nuestros hilos usando Reentrant-Lock en lugar de bloqueos intrínsecos, y podemos usar su método `lockInterruptibly()`:

```
final ReentrantLock l1 = new ReentrantLock();  
final ReentrantLock l2 = new ReentrantLock();
```



```
Thread t1 = new Thread() {  
    public void run() {  
        try {
```

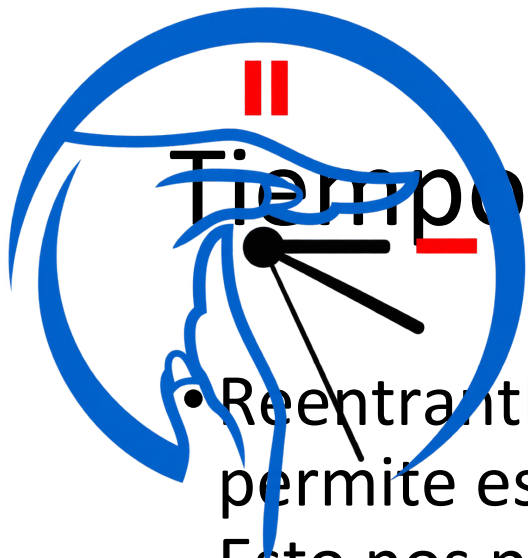
```
        ➤ l1.lockInterruptibly();
```

```
        Thread.sleep(1000);
```

```
        ➤ l2.lockInterruptibly();
```

```
        } catch (InterruptedException e) { System.out.println("t1 interrupted"); }  
    }  
};
```

Esta versión sale limpiamente cuando se llama a `Thread.interrupt()`. La sintaxis un poco más ruidosa de esta versión parece un pequeño precio a pagar por la capacidad de interrumpir un hilo bloqueado.



Tiempos de espera [Timeouts]

- ReentrantLock elimina otra limitación de los cerrojos intrínsecos: nos permite establecer tiempos de espera al intentar adquirir un lock. Esto nos proporciona una forma alternativa de resolver el problema de los “filósofos cenando” presentado el día 1.
- A continuación, hay una versión de philosopher donde el tiempo de espera que se vence si no consigue los dos palillos:

Tiempos de espera [Timeouts]

➤ Jóvenes: Ver lámina 28 del día 1
Ver el código del día 1

```
class Philosopher extends Thread {  
    private ReentrantLock leftChopstick, rightChopstick;  
    private Random random;  
  
    public Philosopher(ReentrantLock leftChopstick, ReentrantLock rightChopstick) {  
        this.leftChopstick = leftChopstick; this.rightChopstick = rightChopstick;  
        random = new Random();  
    }  
    public void run() {  
        try {  
            while(true) {  
                Thread.sleep(random.nextInt(1000)); // Think for a while  
                leftChopstick.lock(); ➡  
                try {  
                    if (rightChopstick.tryLock(1000, TimeUnit.MILLISECONDS)) { ➡  
                        // Got the right chopstick  
                        try {  
                            Thread.sleep(random.nextInt(1000)); // Eat for a while  
                        } finally { rightChopstick.unlock(); } ➡  
                    } else {  
                        // Didn't get the right chopstick - give up and go back to thinking  
                    }  
                } finally { leftChopstick.unlock(); } ➡  
            }  
        } catch (InterruptedException e) {}  
    }  
}
```



Tiempos de espera [Timeouts]

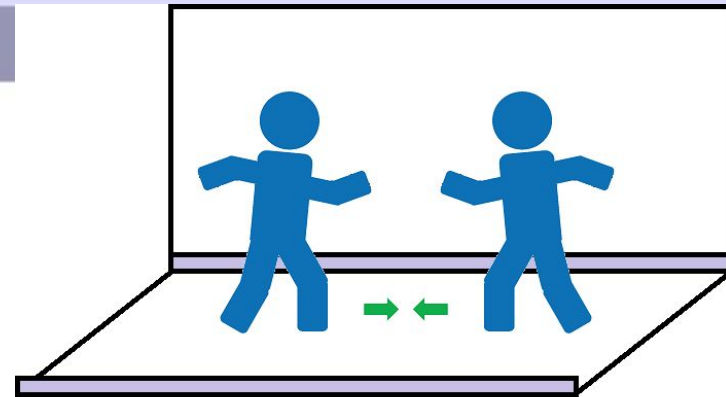
- En lugar de usar `lock()`, el código mostrado antes usa `tryLock ()`, que tiene un tiempo de espera que se vence si no logra adquirir el cerrojo. Esto significa que, aunque no seguimos la regla de "adquirir múltiples bloqueos en un orden global fijo", esta versión no se bloqueará (o al menos, no se bloqueará para siempre).



Livelock

Although the `tryLock()` solution avoids infinite deadlock, that doesn't mean it's a good solution. Firstly, it doesn't avoid deadlock—it simply provides a way to recover when it happens. Secondly, it's susceptible to a phenomenon known as *livelock*—if all the threads time out at the same time, it's quite possible for them to immediately deadlock again. Although the deadlock doesn't last forever, no progress is made either.

This situation can be mitigated by having each thread use a different timeout value, for example, to minimize the chances that they will all time out simultaneously. But the bottom line is that timeouts are rarely a good solution—it's far better to avoid deadlock in the first place.



Bloqueo de mano sobre mano

[Hand-over-Hand Locking]

- Imagine que queremos insertar una entrada en una lista encadenada. Un enfoque sería tener un solo candado [lock] que proteja toda la lista, pero esto significaría que nadie más podría acceder a él mientras mantenemos el candado [lock].
- El bloqueo de mano sobre mano [Hand-over-Hand Locking] es una alternativa en la que bloqueamos solo una pequeña parte de la lista, lo que permite que otros subprocesos tengan acceso sin restricciones siempre que no estén mirando los mismos nodos particulares que tenemos bloqueados. A continuación, se muestra una representación gráfica.
 - El bloqueo mano a mano es un protocolo [patrón] conceptualmente sencillo para el acceso seguro multihilo a estructuras de datos protegidas por bloqueos/locks de grano fino.

Bloqueo mano a mano [Hand-over-Hand Locking]

- El bloqueo mano a mano es una técnica de sincronización fina que evita las condiciones de carrera en datos entre operaciones concurrentes. Comúnmente aplicada a estructuras de datos basadas en punteros, las operaciones bloquean nodos a medida que atraviesan la estructura de datos.
- Para evitar el bypass, la operación propietaria libera el bloqueo de un nodo sólo después de adquirir el bloqueo del siguiente nodo. Por lo general, las operaciones que recorren el mismo camino se ejecutan en cadena. Como el patrón garantiza que dos hilos no accedan simultáneamente a un nodo, se evitan las condiciones de carrera en datos.
- La naturaleza fina del bloqueo mano a mano expone más paralelismo. Dado que cada subproceso bloquea como máximo dos nodos a la vez, varios hilos pueden operar simultáneamente sobre una misma estructura de datos.
- Recibe otros nombre como *lock coupling*, *chain locking*, *latch coupling*, *crabbing*

Bloqueo de mano sobre mano

[Hand-over-Hand Locking]



- Para insertar un nodo, necesitamos bloquear los dos nodos a cada lado del punto que vamos a insertar.
- Comenzamos bloqueando los dos primeros nodos de la lista.
- Si este no es el lugar correcto para insertar el nuevo nodo, desbloqueamos el primer nodo y bloqueamos el tercero.
- Si todavía no es el lugar correcto, desbloqueamos el segundo y bloqueamos el cuarto.
- Esto continúa hasta que encontremos el lugar apropiado, inserte el nuevo nodo y finalmente desbloquee los nodos a cada lado.

Bloqueo de mano sobre mano [Hand-over-Hand Locking]

- Esta secuencia de bloqueos y desbloqueos es imposible con bloqueos intrínsecos(uso de synchronized), pero es posible con ReentrantLock porque podemos llamar a lock() y unlock() cuando queramos. A continuación, hay un ejemplo de una clase que implementa una lista encadenada ordenada usando este enfoque

Line 1 **class** ConcurrentSortedList {

```
- private class Node {  
-     int value;  
5     Node prev;  
-     Node next;  
-     ReentrantLock lock = new ReentrantLock();  
-  
-     Node() {}  
10  
-     Node(int value, Node prev, Node next) {  
-         this.value = value; this.prev = prev; this.next = next;  
-     }  
- }  
15  
- private final Node head;  
- private final Node tail;  
-  
- public ConcurrentSortedList() {  
20     head = new Node(); tail = new Node();  
-     head.next = tail; tail.prev = head;  
- }  
- }
```



```
- public void insert(int value) {  
25     Node current = head;  
-     current.lock.lock();  
-     Node next = current.next;  
-     try {  
-         while (true) {  
30             next.lock.lock();  
-             try {  
-                 if (next == tail || next.value >= value) {  
-                     Node node = new Node(value, current, next);  
-                     next.prev = node;  
35                     current.next = node;  
-                     return;  
-                 }  
-             } finally { current.lock.unlock();  
-                         current = next;  
40                     next = current.next;  
-             }  
-             } finally { next.lock.unlock(); }  
- }  
- }
```



Bloqueo de mano sobre mano

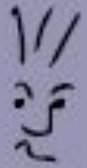
[Hand-over-Hand Locking]

- El método insert() garantiza que la lista siempre se ordene buscando hasta que encuentre la primera entrada que sea menor que la nueva. Cuando lo hace, inserta el nuevo nodo justo antes.
- La línea 26 bloquea el encabezado de la lista y la línea 30 bloquea el siguiente nodo. Luego, verificamos si hemos encontrado el lugar correcto para insertar nuestro nuevo nodo. Si no, el nodo actual se desbloquea en la línea 38 y hacemos un bucle. Si hemos encontrado el lugar correcto, las líneas 33 a 36 crean el nuevo nodo, lo insertan en la lista y regresan. los cerrojos se liberan en los dos últimos bloques (líneas 38 y 42).
- No solo varios subprocesos pueden insertar entradas al mismo tiempo usando este esquema, sino que otras operaciones también pueden tener lugar de forma segura en la lista. Aquí, por ejemplo, hay un método que cuenta cuántos elementos hay en la lista, y solo por diversión, itera hacia atrás a través de la lista:

Bloqueo de mano sobre mano [Hand-over-Hand Locking]

```
public int size() {  
    Node current = tail;   
    int count = 0;  
  
    while (current.prev != head) {  
        ReentrantLock lock = current.lock;  
        lock.lock();   
        try {  
            ++count;  
            current = current.prev;   
        } finally { lock.unlock(); }   
    }  
  
    return count;  
}
```

¿No rompe esto la regla del "orden global"?



Joe asks:

Doesn't This Break the "Global Ordering" Rule?

ConcurrentSortedList's `insert()` method acquires locks starting at the head of the list and moving toward the tail. The `size()` method above acquires them from the tail of the list, moving toward the head. Doesn't this violate the "Always acquire multiple locks in a fixed, global order" rule?

It doesn't, because the `size()` method never holds *multiple* locks—it never holds more than a single lock at a time.

Jóvenes: Ver final lámina 30 del día 1

Una condición para poder utilizar un computador en la sala es que vaya entre las 3:00 pm y las 5:00 pm



Puede ocurrir que llegue a la hora correcta y deba esperar (wait) porque el equipo lo están usando.

No obstante, si voy a las 2:30 pm y el equipo está libre, no puedo utilizarlo porque la condición del horario NO se cumple.

Variables de condición

Variables de condición [Condition Variables]

- La programación concurrente a menudo implica esperar a que suceda algo. Quizás debamos esperar a que una cola no esté vacía antes de eliminar un elemento de ella. O tenemos que esperar a que haya espacio disponible en un búfer antes de agregarle algo. Este tipo de situación es para lo que están diseñadas las variables de condición.
- Para utilizar una variable de condición de forma eficaz, debemos seguir un patrón muy específico:

Variables de condición [Condition Variables]

```
ReentrantLock lock = new ReentrantLock();  
Condition condition = lock.newCondition();
```

```
lock.lock();  
try {  
    while (!«condition is true»)  
        condition.await();  
    «use shared resources»  
} finally { lock.unlock(); }
```



- Una variable de condición está asociada con un cerrojo, y un hilo debe mantener ese bloqueo antes de poder esperar en la condición. Una vez que tiene el cerrojo, comprueba si la condición en la que está interesado ya es cierta. Si es así, continúa con lo que quiera hacer y desbloquea el cerrojo.

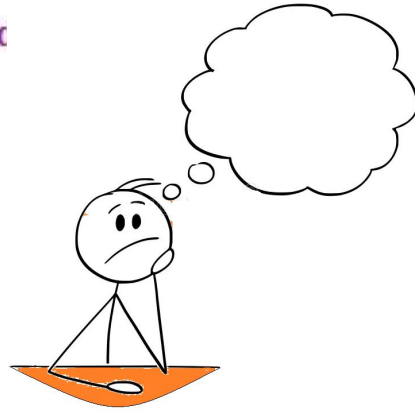
Variables de condición [Condition Variables]

- Sin embargo, si la condición no es verdadera, llama el método `await()`, que desbloquea atómicamente el cerrojo y bloquea la variable de condición. Una operación es atómica si, desde el punto de vista de otro hilo, parece ser una única operación que ha sucedido o no; nunca parece estar a la mitad.
- Cuando otro hilo llama a `signal()` o `signalAll()` para indicar que la condición ahora podría ser verdadera, `await()` desbloquea y vuelve a adquirir automáticamente el cerrojo.
- Un punto importante es que cuando `await()` retorna, solo indica que la condición *podría* ser verdadera. Esta es la razón por la que se llama a `await()` dentro de un bucle: necesitamos volver atrás, volver a verificar si la condición es verdadera y, potencialmente, bloquear en `await()` nuevamente si es necesario.
- Esto nos da otra solución al problema de la cena de los filósofos:

Variables de condición [Condition Variables]

```
class Philosopher extends Thread
```

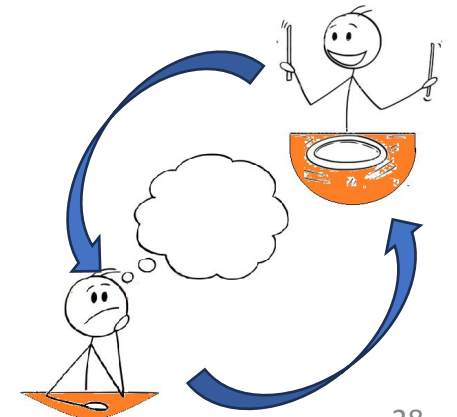
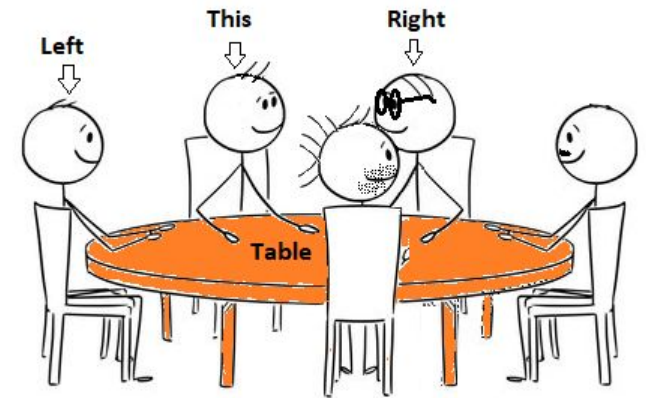
```
private boolean eating;  
private Philosopher left;  
private Philosopher right;  
private ReentrantLock table;  
private Condition condition;  
private Random random;
```



```
public Philosopher(ReentrantLock table) {  
    eating = false;  
    this.table = table;  
    condition = table.newCondition();  
    random = new Random();  
}
```

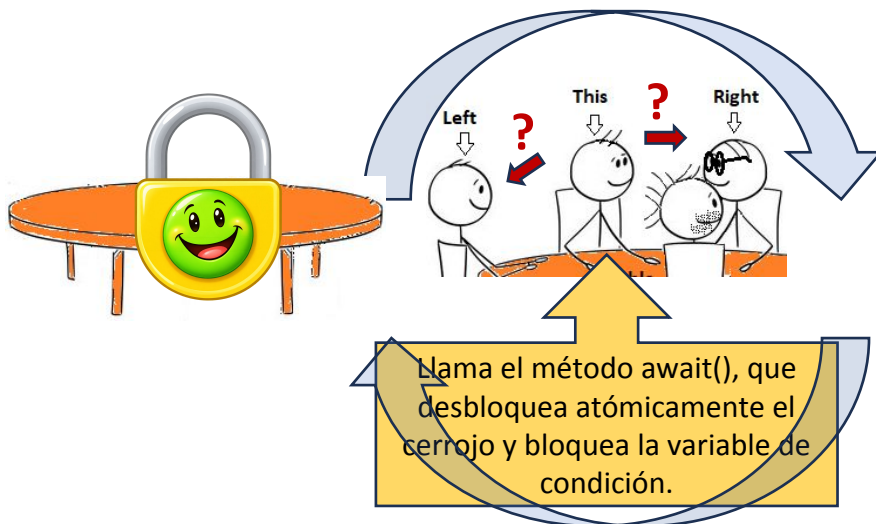
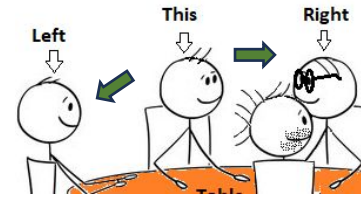
```
public void setLeft(Philosopher left) { this.left = left; }  
public void setRight(Philosopher right) { this.right = right; }
```

```
public void run() {  
    try {  
        while (true) {  
            think();  
            eat();  
        }  
    } catch (InterruptedException e) {}  
}
```



Variables de condición [Condition Variables]

```
private void think() throws InterruptedException {  
    table.lock();  
    try {  
        eating = false;  
        left.condition.signal();  
        right.condition.signal();  
    } finally { table.unlock(); }  
    Thread.sleep(1000);  
}
```

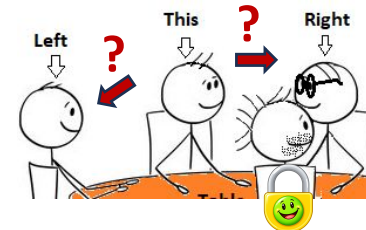


```
private void eat() throws InterruptedException {  
    table.lock();  
    try {  
        while (left.eating || right.eating)  
            condition.await();  
        eating = true;  
    } finally { table.unlock(); }  
    Thread.sleep(1000);  
}
```

Variables de condición [Condition Variables]

- [Además de ser más extensa] Esta solución difiere de las que ya hemos visto al usar un solo cerrojo (table - mesa) y no tener una clase Chopstick explícita. En su lugar, hacemos uso del hecho de que un filósofo puede comer si ninguno de sus vecinos está comiendo actualmente.
- En otras palabras, un filósofo hambriento espera esta condición:

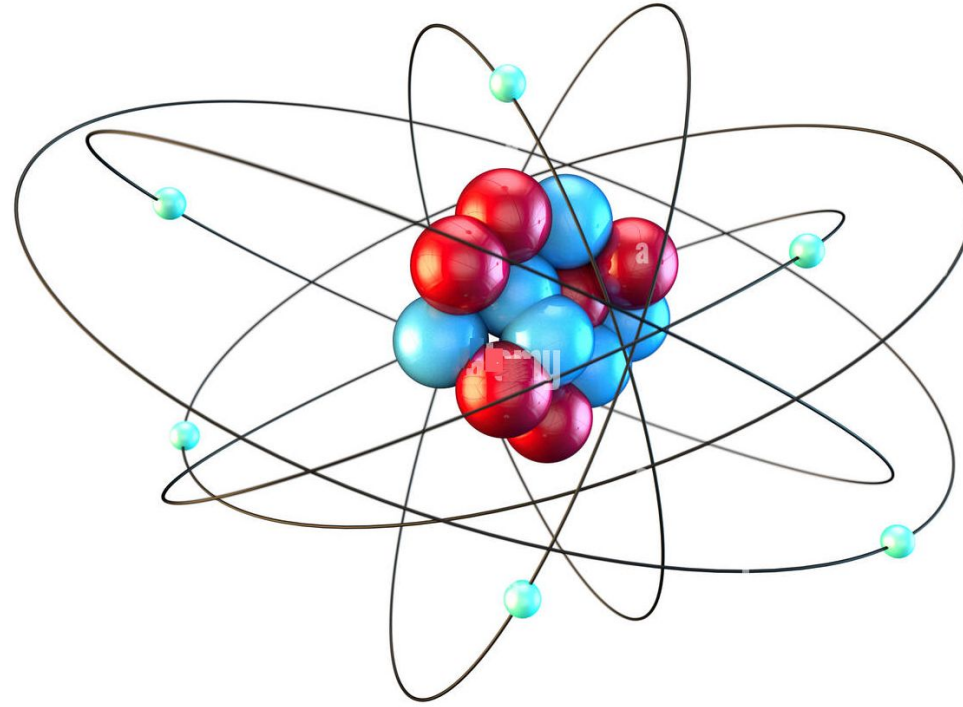
`!(izquierda comiendo || derecha comiendo)`



- Cuando un filósofo tiene hambre, primero “bloquea” la mesa para que ningún otro filósofo pueda cambiar de estado, y luego comprueba si sus vecinos están comiendo actualmente. Si no es así, puede empezar a comer y “desbloquear” la mesa. De lo contrario, llama a `await()` (que desbloquea la mesa).

Variables de condición [Condition Variables]

- Cuando un filósofo ha terminado de comer y quiere empezar a pensar, primero “bloquea” la mesa y pone la variable `eating` en falso. Luego, “envía” una señal a sus dos vecinos para informarles que podrían empezar a comer y “desbloquea” la mesa. Si esos vecinos están esperando en ese momento, los despertará, volverá a adquirir el cerrojo de la mesa y volverá a verificar si pueden comer.
- Aunque este código es más complejo que las otras soluciones que hemos visto, la recompensa es que da como resultado una concurrencia significativamente mejor.
- Con las soluciones anteriores, a menudo ocurre que solo un filósofo hambriento puede comer, porque los demás tienen un solo palillo y esperan que el otro esté disponible. Con esta solución, siempre que sea teóricamente posible que un filósofo coma (cuando ninguno de sus vecinos está comiendo), podrá hacerlo.
- Eso es todo para `ReentrantLock`, pero hay otra alternativa a los bloqueos intrínsecos que cubriremos a continuación: las variables atómicas.



Variables atómicas

Variables atómicas

- El día 1, arreglamos nuestro contador multiproceso sincronizando el método increment() (consulte el código en la página 13). Resulta que java.util.concurrent.atomic proporciona una mejor opción:

```
public class Counting {  
    public static void main(String[] args) throws InterruptedException {  
        ➤ final AtomicInteger counter = new AtomicInteger();  
  
        class CountingThread extends Thread {  
            public void run() {  
                for(int x = 0; x < 10000; ++x)  
                    ➤ counter.incrementAndGet();  
            }  
        }  
  
        CountingThread t1 = new CountingThread();  
        CountingThread t2 = new CountingThread();  
  
        t1.start(); t2.start();  
        t1.join(); t2.join();  
  
        System.out.println(counter.get());  
    }  
}
```



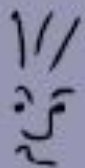
Jóvenes: Ver
láminas 13 y 14
del día 1. Páginas
12 y 13 del libro

Variables atómicas

- El método `incrementAndGet()` de `AtomicInteger` es funcionalmente equivalente a `++count` (también admite `getAndIncrement`, que es equivalente a `count++`). Sin embargo, a diferencia de `++count`, es atómico.
- El uso de una variable atómica en lugar de bloqueos tiene una serie de beneficios. Primero, no es posible olvidarse de adquirir el cerrojo cuando sea necesario.
- Por ejemplo, el problema de visibilidad de la memoria en `Counter`, que surgió porque `getCount()` no estaba sincronizado, no puede ocurrir con este código. En segundo lugar, debido a que no hay cerrojos involucradas, es imposible para una operación con una variable atómica que se bloquee [deadlock].

Variables atómicas

- Finalmente, las variables atómicas son la base de los algoritmos sin bloqueo (non-blocking) y libres de locks (lock-free), que logran la sincronización sin locks ni bloqueos.
- Si considera que programar con cerrojos es complicado, espere hasta que intente escribir un código sin bloqueos [lock-free].
- Afortunadamente, las clases en `java.util.concurrent` hacen uso de código sin bloqueo [lock-free] siempre que sea posible, para que pueda aprovecharlo sin problemas. Cubriremos [estas clases de java] en el día 3.



Joe asks:

What About Volatile?

Jóvenes: Ver lámina 17 del día 1.

Java allows us to mark a variable as *volatile*. Doing so guarantees that reads and writes to that variable will not be reordered. We could fix Puzzle (see the [code on page 13](#)) by making `answerReady` volatile.

Volatile is a very weak form of synchronization. It would not help us fix Counter, for example, because making `count` volatile would not ensure that `count++` is atomic.

These days, with highly optimized JVMs that have very low-overhead locks, valid use cases for volatile variables are rare. If you find yourself considering volatile, you should probably use one of the `java.util.concurrent.atomic` classes instead.

Resumen día 2

- Nos hemos basado en los conceptos básicos presentados en el día 1 para cubrir algunos mecanismos más sofisticados y flexibles proporcionados por `java.util.concurrent.locks` y `java.util.concurrent.atomic`.
- Aunque es importante comprender estos mecanismos, rara vez se usarán bloqueos directamente en la práctica, como veremos en el día 3.
- Vimos cómo `ReentrantLock` y `java.util.concurrent.atomic` pueden superar las limitaciones de los bloqueos intrínsecos para que nuestros hilos puedan hacer lo siguiente:
 - Ser interrumpidos al intentar adquirir un cerrojo
 - Utilizar tiempos de espera [timeouts] al adquirir un cerrojo
 - Adquirir y liberar cerrojos en cualquier orden
 - Utilizar variables de condición para esperar a que se cumplan condiciones “arbitrarias”
 - Evitar el uso de cerrojos por completo mediante el uso de variables atómicas

Autoestudio día 2

- ReentrantLock admite un parámetro de equidad [fairness parameter]. ¿Qué significa que un cerrojo [lock] sea "justo" [fair]? ¿Por qué elegiría un cerrojo [lock] justo [fair]? ¿Por qué no se podría utilizar?
- ¿Qué es ReentrantReadWriteLock? ¿En qué se diferencia de ReentrantLock? ¿Cuándo se puede usar?
- ¿Qué es un "despertar espurio" ["spurious wakeup"]? ¿Cuándo puede suceder y por qué no le importa a un programa bien redactado si lo hace?
- ¿Qué es AtomicIntegerFieldUpdater? ¿En qué se diferencia de AtomicInteger? ¿Cuándo se puede usar?

Para hacer

- ¿Qué pasaría si el bucle dentro de la implementación de los "filósofos de la cena" que usa variables de condición, fuera reemplazado por una simple declaración if? ¿Qué modos de falla puede ver? ¿Qué pasaría si la llamada a signal() fuera reemplazada por signalAll()? ¿Qué problemas (si los hay) causaría esto?
- Así como los bloqueos [locks] intrínsecos son más limitados que ReentrantLock, también admiten un tipo más limitado de variable de condición. Reescriba los filósofos de la cena para usar un bloqueo [lock] intrínseco más los métodos wait() y notify() o notifyAll (). ¿Por qué es menos eficiente que usar ReentrantLock?
- Escriba una versión de ConcurrentSortedList que utilice un solo bloqueo [lock] en lugar de un bloqueo [lock] manual. Compárelo con la otra versión. ¿El bloqueo mano sobre mano proporciona alguna ventaja de rendimiento? ¿Cuándo podría ser una buena elección? ¿Cuándo podría no ser así?

Programas revisados

1.	Counting	Ok	
2.	CountingBetter	Ok	
3.	CountingFixed	Ok	
4.	DiningPhilosophers	Ok	
5.	DiningPhilosophersCondition	Ok	
6.	DiningPhilosophersFixed	Ok	
7.	DiningPhilosophersTimeout	Ok	
8.	EchoServer	NO	
9.	EchoServerBetter	NO	
10.	HelloWorld	Ok	
11.	HttpDownload	Ok	
12.	HttpDownloadBetter	NO	
13.	HttpDownloadFixed	Ok	
14.	Interruptible	Ok	
15.	LinkedList	Ok	
16.	Puzzle	Ok	
17.	Uninterruptible	Ok	
18.	WordCount	NO	
19.	WordCountBatchConcurrentHashMap	NO	
20.	WordCountConcurrentHashMap	NO	
21.	WordCountProducerConsumer	NO	
22.	WordCountSynchronizedHashMap	NO	

Gracias